Processor Verification
Lab 5 - Coverage

Lorenzo Pattaro Zonta

May 7, 2026

Introduction

Design Under Test - FIFO queue

The design under test (DUT) is a simple FIFO module with the following interface:

Signal	Direction	Width	Description
clk	input	1 bit	Clock signal
rst_n	input	1 bit	Active-low reset signal
wr_en	input	1 bit	Write enable signal
rd_en	input	1 bit	Read enable signal
wr_data	input	8 bits	Data to be written into the FIFO
rd_data	output	8 bits	Data read from the FIFO
empty	output	1 bit	Indicates if the FIFO is empty
full	output	1 bit	Indicates if the FIFO is full
count	output	4 bits	Number of elements currently in the FIFO

Behavioral description

The FIFO module implements a first-in-first-out data structure. It has a depth of 16 and a width of 8 bits. The module operates as follows:

- On the rising edge of the clock, if the reset signal is active (low), the FIFO is cleared, setting the count to 0 and the empty flag to 1.
- If the write enable signal is high and the FIFO is not full, the data on the wr_data input is written into the FIFO, the count is incremented, and the empty flag is cleared.
- If the read enable signal is high and the FIFO is not empty, the data at the front of the FIFO is output on the rd_data output, the count is decremented, and if the count reaches 0, the empty flag is set.
- The full flag is set when the count reaches 16, indicating that the FIFO cannot accept more data until some data is read out.

Task 1

Question 1

The 13 covergroup bins are:

- Explicit (9): `went_full`, `went_empty`, `empty_bin`, `mid_bins`, `full_bin`, `write_only`, `read_only`, `both_off`, `illegal`

- Auto (4): `wr_en=0`, `wr_en=1`, `rd_en=0`, `rd_en=1`

The directive is `cover property (becomes_empty_after_read)`.

Question 2

Assuming reset keeps `empty=1`, `full=0`, `dut.count=0`, and `wr_en=rd_en=0`, the bins covered by reset alone, with the stimulus commented (reset + idle only), are:

- `went_empty`, `empty_bin`
- `wr_en=0` (auto), `rd_en=0` (auto)
- `both_off`

Task 2

Question 1

With the stimulus uncommented (5 writes, 10 idle, 2 reads), these bins get covered:

- `went_empty`, `wr_en` (both 0 and 1), `rd_en` (both 0 and 1)
- `empty_bin` and `mid_bins` (of coverpoint count)
- for cross coverage of `wr_en` and `rd_en` : `write_only`, `read_only` and `both_off`

In either cases (commented stimulus and uncommented one) these are not covered:

- `went_full`, `full_bin` (of coverpoint count), `illegal` (of cross coverage of `wr_en` and `rd_en`)
- `cover property (becomes_empty_after_read)`

Task 3

The following stimulus has been added to the testbench to cover the missing bins and reach a 100% covergroup coverage and 100% directive coverage:

```
repeat (4) begin
  @(posedge clk);
  wr_en = 0;
  rd_en = 1;
end

repeat (16) begin
  @(posedge clk);
  wr_en = 1;
  rd_en = 0;
  wr_data = $urandom_range(0,255);
end

@(posedge clk);
wr_en = 1;
rd_en = 1;
```

The first stimulus covers the bins `went_empty`, `empty_bin` and the directive `cover property (becomes_empty_after_read)` because the dut is emptied and the empty flag is set when the last value is read and the FIFO becomes empty. The second stimulus covers the `went_full` and `full_bin` bins because we fully populate the FIFO, with 16 writes, and the last stimulus covers the `illegal` bin.ì, as we try to write and read at the same time, which is not allowed by the design.

Design Under Test - Direct Mapped Cache

The design under test (DUT) is a direct mapped cache module with the following parameteres and interface:

Parameter	Description	Default Value
CACHE_LINES	Number of cache lines (sets)	16
LINE_SIZE	Number of words per line (block size)	4
ADDR_WIDTH	Width of the address bus in bits	8
DATA_WIDTH	Width of each data word in bits	32

Signal	Direction	Width	Description
clk	input	1 bit	Clock signal
reset	input	1 bit	Asynchronous reset (active high)
read	input	1 bit	Read enable signal
write	input	1 bit	Write enable signal
addr	input	ADDR_WIDTH	Address of the access
data_in	input	DATA_WIDTH	Data to be written to the cache
data_out	output	DATA_WIDTH	Data returned from the cache on read
hit	output	1 bit	Indicates if the access was a cache hit (1) or a miss (0)

The internal structure of the cache includes:

- Cache memory: a 2D array `cache_mem[CACHE_LINES][LINE_SIZE]` that stores the data words for each cache line.
- Tag array: a 1D array `tag_array[CACHE_LINES]` that stores the tag for each cache line, used to identify which block of memory is currently stored in that line.
- Valid array: a 1D array `valid_array[CACHE_LINES]` that indicates whether the data in each cache line is valid (1) or not (0).

The addess is split into three fields:

- Tag (addr[ADDR_WIDTH-1 -: TAG_WIDTH]): the most significant bits of the address, used to identify the block of memory stored in the cache line.
- Index (addr[OFFSET_WIDTH+INDEX_WIDTH-1 -: INDEX_WIDTH]): the middle bits of the address, used to select the cache line (set) being accessed.
- Offset (addr[OFFSET_WIDTH+INDEX_WIDTH-1 -: INDEX_WIDTH]): the least significant bits of the address, used to select the specific word within the cache line.

Behavioral Description

The cache operates as follows: upon reset, all valid bits are cleared, all tags and cache memory words are set to zero, and the outputs (hit and data_out) are cleared. During a read operation, if the valid bit for the indexed cache line is set and the tag matches, it is a hit, and the data is returned from the cache. If it is a miss, the hit signal is cleared, a simulated memory fetch returns data based on the address, and the cache line is updated with the new data, tag, and valid bit. During a write operation, if there is a hit (valid bit set and tag matches), the hit signal is set; otherwise, it is a miss. In both cases of a write operation, the cache memory at the indexed line and offset is updated with the input data. If neither read nor write is asserted, the outputs remain stable with hit cleared.

Limitations

The design has some limitations and simplifications: read operations take precedence over write operations, meaning that if both are requested simultaneously, only the read operation will be executed. There is no replacement policy since it is a direct-mapped cache, which always overwrites the line at the indexed location. Additionally, there is no write-back or write-through policy implemented; writes only update the cache without modeling any backing memory. Miss handling is simplified by returning a value based on the address instead of fetching from real memory. Finally, there are no stall or wait states; all operations complete in one cycle.

Task 4

I have implemented the following covergroups and coverpoints in the testbench. I have created two covergroups: `cache_cov` for general coverage of the cache behavior and `cache_cov_reset` specifically for coverage related to the reset behavior, to be sure that we do not obtain false positives in the coverage results due to the reset state of the cache. With the provided random test, the total covergroup coverage is 97.93%, with the covergroup `cache_cov` at 95.87% and the covergroup `cache_cov_reset` at 100%. The directive coverage is 100%, as both properties are covered by the random stimulus. The toggle coverage is 99.41%, with the only missing toggle being the one of the hit signal when both read and write are asserted, which is not possible in our design as read takes precedence over write.

```
covergroup cache_cov @(posedge clk iff !reset);
    option.per_instance = 1;
    option.name = "Coverage for simple_cache";

    tag_bin: coverpoint dut.tag {
        bins tag = {[dut.TAG_WIDTH-1:0]};
    }

    index_bin: coverpoint dut.index {
        bins index = {[dut.INDEX_WIDTH-1:0]};
    }

    offset_bin: coverpoint dut.offset {
        bins offset = {[dut.OFFSET_WIDTH-1:0]};
    }

    coverpoint hit {
        bins hit_bin = {1'b1};
        bins miss_bin = {1'b0};
    }

    validity: coverpoint dut.valid_array[dut.index] {
        bins valid_bin = {1'b1};
        bins invalid_bin = {1'b0};
    }

    coverpoint write;
    coverpoint read;

    read_write: cross read, write {
        bins read_only  = binsof(read) intersect {1'b1} && binsof(write) intersect {1'b0};
        bins write_only = binsof(read) intersect {1'b0} && binsof(write) intersect {1'b1};
        bins read_precedence    = binsof(read) intersect {1'b1} && binsof(write) intersect
        ↪  {1'b1};
        bins nothing    = binsof(read) intersect {1'b0} && binsof(write) intersect {1'b0};
    }
```

```
    hit_miss_cross: cross hit, valid_accessed {
        bins hit_valid   = binsof(hit) intersect {1'b1} && binsof(valid_accessed) intersect
        ↪  {1'b1};
        bins miss_valid  = binsof(hit) intersect {1'b0} && binsof(valid_accessed) intersect
        ↪  {1'b1};
        bins hit_invalid = binsof(hit) intersect {1'b1} && binsof(valid_accessed) intersect
        ↪  {1'b0};
        bins miss_invalid= binsof(hit) intersect {1'b0} && binsof(valid_accessed) intersect
        ↪  {1'b0};
    }

    read_write_prev_hit: cross read_prev, write_prev, hit {
        bins read_nowrite_hit       = binsof(read_prev) intersect {1'b1} && binsof(write_prev)
        ↪  intersect {1'b0} && binsof(hit) intersect {1'b1};
        bins read_nowrite_miss      = binsof(read_prev) intersect {1'b1} && binsof(write_prev)
        ↪  intersect {1'b0} && binsof(hit) intersect {1'b0};
        bins noread_write_hit      = binsof(read_prev) intersect {1'b0} && binsof(write_prev)
        ↪  intersect {1'b1} && binsof(hit) intersect {1'b1};
        bins noread_write_miss     = binsof(read_prev) intersect {1'b0} && binsof(write_prev)
        ↪  intersect {1'b1} && binsof(hit) intersect {1'b0};
        bins read_write_hit  = binsof(read_prev) intersect {1'b1} && binsof(write_prev)
        ↪  intersect {1'b1} && binsof(hit) intersect {1'b1};
        bins read_write_miss = binsof(read_prev) intersect {1'b1} && binsof(write_prev)
        ↪  intersect {1'b1} && binsof(hit) intersect {1'b0};
        bins nothing                = binsof(read_prev) intersect {1'b0} && binsof(write_prev)
        ↪  intersect {1'b0} && binsof(hit) intersect {1'b0};
        illegal_bins not_possible   = binsof(read_prev) intersect {1'b0} && binsof(write_prev)
        ↪  intersect {1'b0} && binsof(hit) intersect {1'b1};
    }

endgroup

covergroup cache_cov_reset @(posedge clk);
    option.per_instance = 1;
    option.name = "Coverage for simple_cache - reset behavior";

    reset_cp: coverpoint dut.reset {
        bins reset_active = {1'b1};
        bins reset_inactive = {1'b0};
    }
endgroup

// miss on invalid line
property miss_on_invalid;
    @(posedge clk) disable iff (reset) (read && !dut.valid_array[dut.index]) |-> (hit == 0);
endproperty

// hit on valid line with matching tag
property hit_on_valid;
    @(posedge clk) disable iff (reset) (read && dut.valid_array[dut.index] &&
    ↪  dut.tag_array[dut.index] == dut.tag) |-> (hit == 1);
endproperty
```

Task 5

The following stimulus has been added to the testbench to fully cover the toggle coverage and the remaining bins of the covergroups. For the toggle coverage, the missing toggle was the reset from 0 to 1, which is not

possible in the provided stimulus as the reset is only asserted at the beginning of the test and then deasserted. So, I have added a reset pulse in the middle of the test (`#20 reset = 1; #20 reset = 0;`). The first missing bins of the covergroups was "read_precedence" of the read_write cross coverage, which is hit when both read and write are asserted at the same time, so I have added a stimulus that asserts both read and write at the same time (`@(posedge clk); read = 1; write = 1;`). I've also added addr = 8'hA5; because there was also the bins "read_write_hit" and "read_write_miss" of the read_write_prev_hit cross coverage, which require a read hit and a read miss when both read and write are asserted at the same time. The first time we obtain a read miss because the line is invalid, and the second time we obtain a read hit because the line is now valid and contains the data we wrote in the previous cycle.

```
@(posedge clk);
        reset = 1;
        #20 reset = 0;

        @(posedge clk); // read miss on invalid line
        read = 1; write = 1; addr = 8'hA5; //

        @(posedge clk);
        read = 1; write = 1; addr = 8'hA5; // read hit on valid line
```

Thanks to these additional stimuli, I have obtained a 100% toggle coverage, as well as 100% covergroup coverage and 100% directive coverage.

1 Appendix

1.1 Number of hours required

For the completion of this laboratory, it has been needed 10 hours.

1.2 Use of AI

Correction of some bugs, minor text corrections, clarifications and explanations.